



Bangladesh Army University of Science and Technology
Saidpur, Nilphamari

Project Title: Smart City Transport Management System

SESSIONAL PROJECT REPORT

CSE 2206

Database Management Systems Sessional

Submitted By

Student Name	Student ID
1. Md.Al-Maynul Hasan	0802420205101025
2. Md.Asif Hasan	0802420205101032
3. Md.Maruf Siddiki	0802420205101034

Date: 01 June 26

Submitted To:

AKZ Rasel Rahman

Lecturer

Department of Computer Science and Engineering

Bangladesh Army University of Science and Technology

S. M Golam Rifat

Lecturer

Department of Computer Science and Engineering

Bangladesh Army University of Science and Technology

The complete source code of this project is available at: <https://github.com/AlMaynulHasan/SCTMS>

Table of Contents

Contents

Database Management Systems Sessional.....	1
Submitted By	1
Table of Contents	3
Introduction	1
1.1 Introduction	1
1.2 Objectives.....	1
1.3 Project Outcome	1
System Analysis & Design	2
2.1 Requirement Analysis & Design Specification	2
2.2 Overall Project Plan.....	3
Implementation and Results.....	4
3.1 UI Design.....	4
3.2 Workflow of Application	6
3.3 Code Snippets (Database connection)	7
3.4 Results and Discussion	9
Conclusion.....	9
References.....	10

Chapter 1

Introduction

1.1 Introduction

Urban transportation systems across developing countries face persistent challenges in managing bus and train schedules, passenger information, ticket booking, and real-time seat availability. Traditional manual or semi-digital systems frequently result in booking conflicts, inaccurate seat counts, duplicate ticket issuance, and inefficient data retrieval. Passengers are often left without reliable information about available seats, schedule timings, or pricing, leading to frustration and loss of productivity.

The Smart City Transport Management System (SCTMS) is a full-stack, web-based database application designed to solve these problems. SCTMS enables users to search for routes, book tickets, make payments, exchange tickets with other passengers, and view schedules in real time. The system includes a dedicated admin panel for managing routes, schedules, users, and promotional codes, making it a comprehensive transport management solution. The system was developed as a sessional project for CSE 2206 — Database Management Systems, demonstrating the practical integration of a relational database with a full-stack web application.

1.2 Objectives

1. To design a fully normalized relational database using Microsoft SQL Server, covering all aspects of transport management.
2. To develop a web-based ticket booking system supporting multi-seat selection and seat-map visualization.
3. To implement secure user authentication and role-based access control using JWT.
4. To automate fare calculation and real-time seat availability updates using database stored procedures and triggers.
5. To enable a peer-to-peer ticket exchange marketplace allowing passengers to sell or swap their tickets securely.
6. To build a fully functional admin dashboard for route, schedule, promo code, and user management.

1.3 Project Outcome

Upon completion, the Smart City Transport Management System provides the following outcomes:

- A normalized SQL Server database managing users, routes, schedules, bookings, payments, and ticket transfers.
- A passenger-facing web interface for route search, seat selection, booking, and ticket exchange.
- An admin dashboard for managing the entire transport ecosystem.
- Stored procedures for automated fare calculation and promo code validation.
- Database triggers for real-time seat availability management.
- A printable ticket page with unique booking IDs per seat.
- A fully documented REST API with 20+ endpoints.

Chapter 2

System Analysis & Design

2.1 Requirement Analysis & Design Specification

2.1.1 Overview

The Smart City Transport Management System (SCTMS) is designed to provide a user-friendly and efficient full-stack solution for managing urban public transportation logistics, passenger ticket bookings, and peer-to-peer ticket trading. The system captures essential transactional data, including user profiles, routes, trip schedules, seat reservations, and payment processing records, storing it in a centralized, highly optimized relational database for seamless real-time data retrieval.

Key functional requirements of the system include:

- Schedule Search: Query real-time bus/train schedules by origin, destination, and date.
- Seat-Map Selection: Interactive grid allowing multi-seat selection with real-time seat locking.
- Automated Fare Engine: Instant fare computation with automated promo code validation.
- P2P Marketplace: Peer-to-peer platform for passengers to safely resell or swap tickets.
- Waitlist Queue: Automatic queueing and prioritization for sold-out trips.
- Admin Dashboard: Central console to manage routes, schedules, users, and platform data.
- Ticket Generation: Simulates OTP payment and outputs printable tickets with unique seat IDs.

Key non-functional requirements include:

- Performance: Sub-second response times using database indexing and real-time triggers.
- Usability: Responsive multi-page web UI featuring native English/Bengali localization toggles.
- Reliability: Strict 3NF database normalization ensuring high data integrity and zero conflicts.
- Security: Role-based access control secured via JWT and hashed passenger passwords (bcrypt).
- Maintainability: Decoupled three-tier architecture (HTML/JS → Node.js API → MS SQL Server).

2.1.2 Project Development Methodology Flowchart

The Smart City Transport Management System (SCTMS) was developed using a structured, hybrid software development methodology. This framework combines the predictive, clear-phased planning of the Waterfall Model with the flexible, evolutionary adjustments of the Iterative Approach.

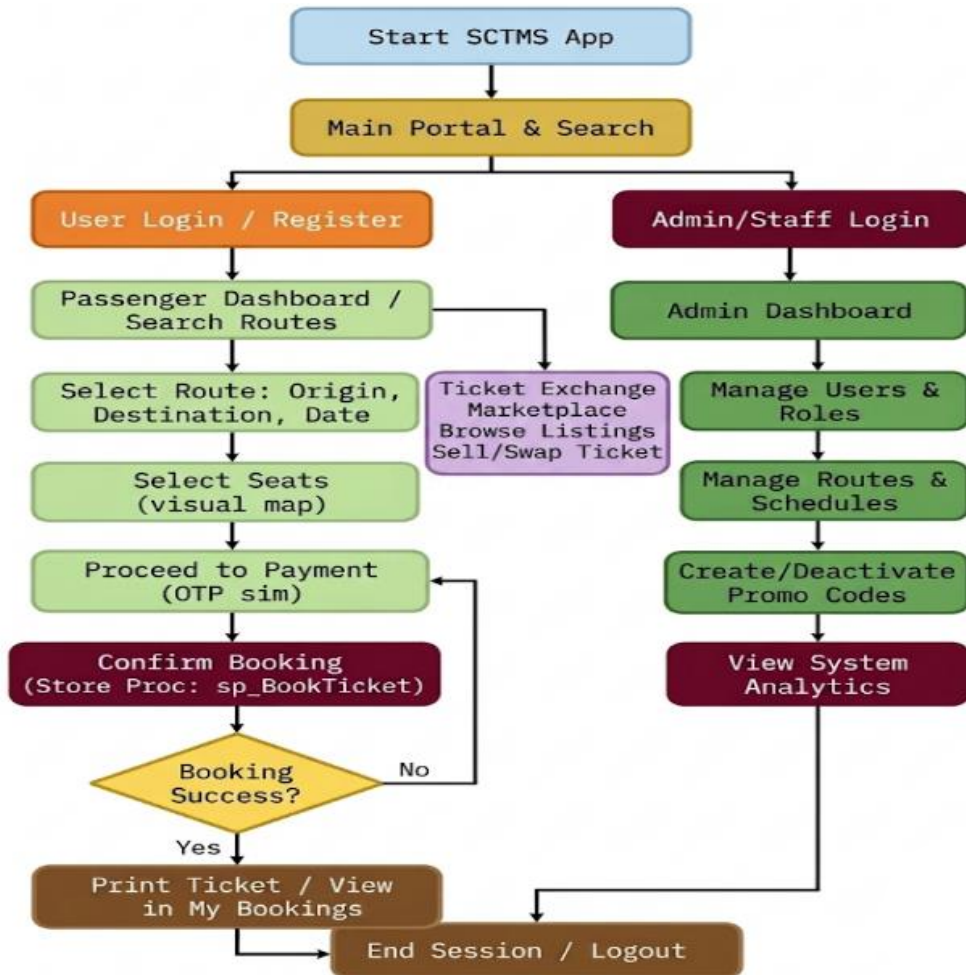


Figure 2.1: SCTMS Hybrid Software Development Methodology Flowchart

2.2 Overall Project Plan

Phase	Tasks / Activities	Estimated Duration
Requirement Analysis	Collect public transit use-cases; define core passenger booking rules and marketplace requirements; establish clear functional and non-functional engineering specifications.	1 week
System Design	Design fully normalized relational schemas up to 3NF; define key database entities, primary/foreign key relationships, stored procedures, triggers, and the core 3-tier system architecture layout.	1 week
Implementation	Program the Node.js backend server API endpoints; implement JWT user authentication; construct the responsive frontend UI web pages using raw HTML5, CSS3, and JavaScript.	2–3 weeks
Testing	Validate full end-to-end user transactional paths (Search → Map Selection → Booking → Simulated OTP Payment); audit database trigger execution for accurate seat counters; fix cross-origin resource sharing (CORS) bugs.	1 week
Deployment & Maintenance	Export the production-ready SQL relational schema backup files; compile the technical documentation report; push clean system source files to the remote repository for evaluation.	1 week

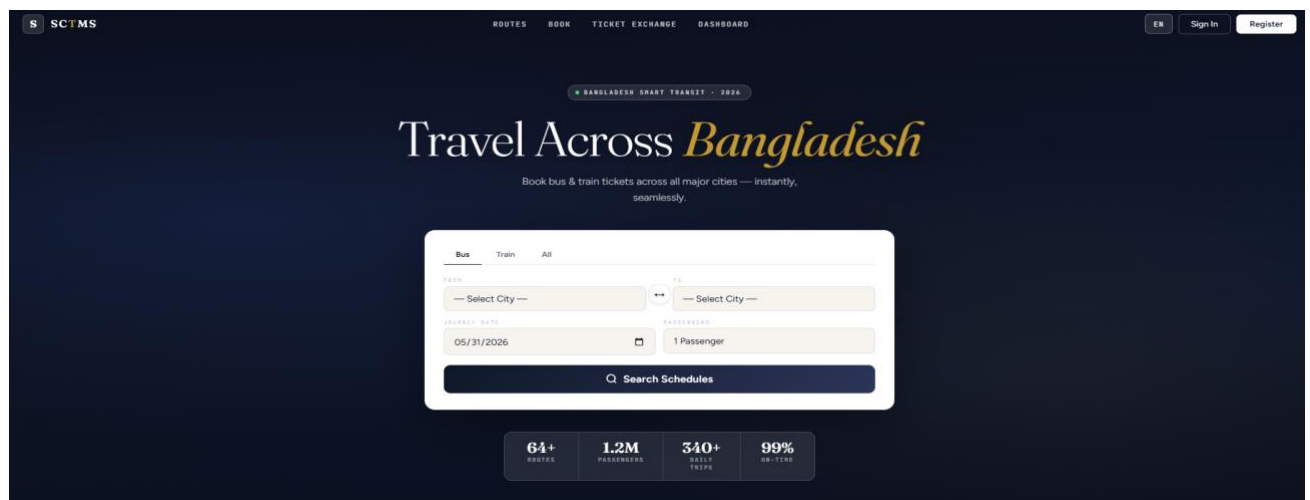
Chapter 3

Implementation and Results

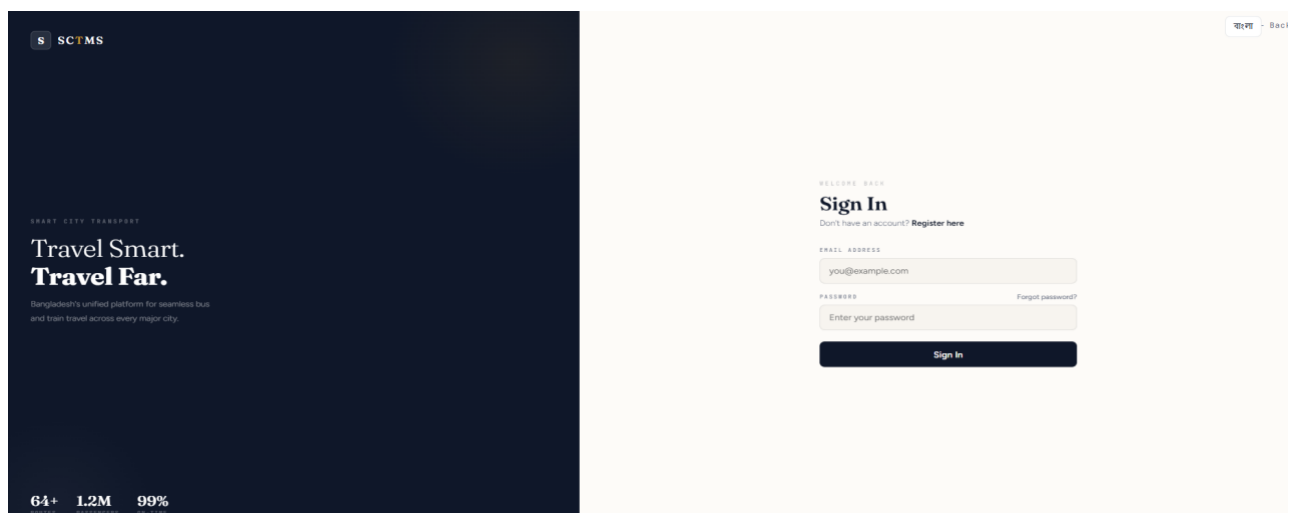
3.1 UI Design

The system features a fully responsive, web-based user interface developed using HTML5, CSS3, and JavaScript to provide a seamless, modern, and user-friendly experience for transit search and ticket management. The application's layout is completely browser-rendered and consists of multiple interactive pages and reusable web components:

- **Dashboard Interface.**



- **Login Interface (User Authentication)**



- **Ticket Booking Interface (Route Search & Seat Selection).**

The interface is divided into two main sections. The left section, titled "Search & Book", contains a search form with fields for "FROM" (Dhaka), "TO" (Chittagong), "JOURNEY DATE" (05/31/2026), and "LRL_PASS" (1 Passenger). Below the form, it shows "Showing 3 results Dhaka -> Chittagong". Three train options are listed:

- Mahanagar Provti, Dhaka -> Chittagong, 07:00 DEPARTURE, 07:00 ARRIVAL, BDT 420, Select Seat.
- Mahanagar Provti, Dhaka -> Chittagong, 14:00 DEPARTURE, 14:00 ARRIVAL, BDT 420, Select Seat.
- Mahanagar Provti, Dhaka -> Chittagong, 22:00 DEPARTURE, 22:00 ARRIVAL, BDT 580, Select Seat.

 Each option indicates "AC Standard Seat" and "350 seats left". The right section, titled "Select Seat", shows a grid of seats for the selected train. The price is "BDT 420 per seat". The grid shows rows 1-13 and columns A1-A3, B1-B3, C1-C3. The "Selected Seat" field is empty, and the "Continue to Payment" button is visible at the bottom.

- **Ticket View Interface (My Bookings / Print Ticket)**

The interface is divided into two main sections. The left section, titled "My Bookings", shows a list of bookings with columns for "Status", "Train", "Date", and "Price". The right section, titled "Print Ticket", shows a detailed view of a specific ticket. The ticket is for the "Dhaka -> Chittagong" route, dated "31 May 2026", with a price of "BDT 430". It includes a QR code and a "Download / Print" button.

- **Ticket Exchange Module (Special Feature).**

The interface is divided into two main sections. The left section, titled "Swap your ticket, skip the cancellation fee", explains the process of swapping a ticket. The right section, titled "List a Ticket", shows a form for listing a ticket for exchange. The form includes fields for "Train" (Dhaka -> Chittagong), "Date" (31 May 2026), and "Price" (BDT 430). It also includes a "Smart Price Suggestion" section with a "Suggested price" of BDT 430 and a "Post Listing" button.

3.2 Workflow of Application

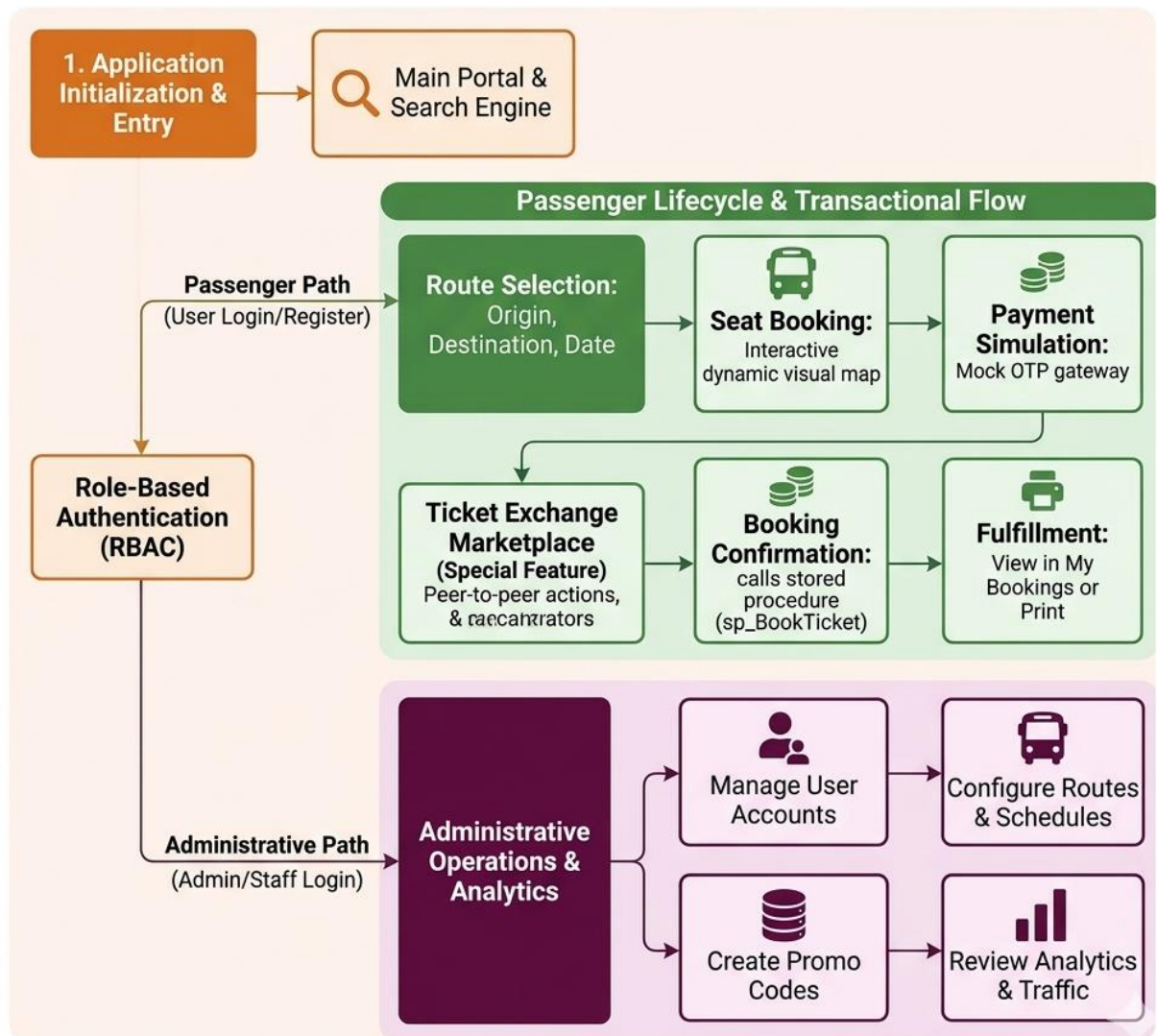


Figure: Smart City Transport Management System Workflow Diagram

3.3 Code Snippets (Database connection)

```
const sql = require('mssql');
require('dotenv').config();
const config = {
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  server: process.env.DB_SERVER || 'localhost',
  database: process.env.DB_NAME,
  port: parseInt(process.env.DB_PORT || 1433),
  options: {
    encrypt: true,
    trustServerCertificate: true
  }
};
const poolPromise = new sql.ConnectionPool(config)
  .connect()
  .then(pool => {
    console.log(' MSSQL Database Connected Successfully');
    return pool;
  })
  .catch(err => {
    console.error('Database Connection Failed:', err.message);
    process.exit(1); // Exit if DB fails
  });
module.exports = { poolPromise, sql };
// After all routes and middleware
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log('\n;
  console.log ( ' SCTMS Backend v2.1.0 ');
  console.log ( ` Server  → http://localhost:${PORT}`);
  console.log ( 'Port 5500 & 5502 allowed ');
  console.log ( 'AutoSched 14-day rolling window');
  console.log ('\n');
});
```

3.4 Results and Discussion

The Smart City Transport Management System (SCTMS) successfully implements a full-stack web solution optimizing public transit logistics, ticket bookings, and peer-to-peer (P2P) trading. Passengers can query routes by origin, destination, and date to access live schedules, select seats from an interactive grid, apply promo codes, and process simulated payments to generate individual travel vouchers. A core innovation is the P2P ticket marketplace. Storing transactions on a granular row-per-seat model allows passengers to resell or swap individual seats directly from their dashboard without canceling an entire multi-seat travel itinerary. Simultaneously, a centralized Admin Dashboard provides administrators full control to manage users, modify schedules, deploy promo codes, and monitor analytics. From a data perspective, normalizing the Microsoft SQL Server database schema to Third Normal Form (3NF) eliminated redundancies and ensured total referential integrity. Performance bottlenecks were removed by applying relational indexing to high-traffic columns (Email, ScheduleID, RouteID), enabling complex multi-table JOIN sequences to execute in sub-second timelines. Furthermore, critical transactional workflows—including real-time seat inventory count updates and promotional fare deductions—are handled directly inside the data tier via automated SQL triggers and stored procedures. This configuration minimizes application-to-database communication delays, completely preventing duplicate seat allocation or race conditions during high-concurrency booking traffic. Ultimately, the project delivers a scalable, highly secure, and paperless transport infrastructure. Future updates will transition the local system to production by embedding live digital payment gateways (e.g., SSLCommerz, bKash), integrating GPS-based vehicle tracking, and deploying native cross-platform mobile interfaces.

Chapter 4

Conclusion

The Smart City Transport Management System (SCTMS) is a comprehensive full-stack web application that provides an efficient, integrated solution for managing urban public transportation logistics, processing seat reservations, and facilitating passenger ticket trading. By using a decoupled three-tier architecture, users can smoothly query live travel routes, select available seats from an interactive visual map grid, apply discount promotional codes, and instantly generate individual travel passes in a structured and secure manner. The system ensures transactional data accuracy and structural consistency through strict Third Normal Form (3NF) relational normalization, server-side validation, and automated database triggers and stored procedures. The inclusion of a centralized Admin Dashboard allows transportation authorities to manage transit logistics effectively, offering features like route instantiation, dynamic schedule planning, and macro system analysis. The integrated peer-to-peer (P2P) ticket exchange marketplace introduces a novel, user-driven ecosystem where verified passengers can legally resell or swap their individual seat bookings with alternative travelers. Overall, the project successfully achieves its primary objectives of data integrity, scalability, and system performance. It demonstrates how a Microsoft SQL Server relational database, a Node.js API layer, and an optimized native frontend can be cleanly combined to build a lightweight, highly functional transport infrastructure application.

Limitations

- **Web-Only Client Environment:** The platform operates exclusively inside web browsers, limiting mobile access and missing dedicated native Android or iOS application optimization.
- **Simulated Financial Environment:** The payment confirmation logic utilizes a mock OTP simulation system rather than linking to production digital banking networks (e.g., bKash, Nagad, SSLCommerz).
- **Polled Notifications Architecture:** In-app notification changes depend on traditional pulled database queries executed at page refresh rather than persistent, live bi-directional WebSockets.
- **Absence of Real-Time Telemetry:** The application functions without live vehicle physical positioning matrix tracking or real-time GPS telemetry overlays.
- **Development Email Framework:** The built-in account verification and verification code routines are configured locally but require active production SMTP credentials to email external servers.
- **Single-Process Application Instance:** The application backend processes business computations on a single Node.js runtime instance without active process clustering, caching layers, or advanced load balancing configuration.

Future Extensions

- Production Gateway Integrations: Connect the application transaction pipelines to authorized financial payment aggregators like SSLCommerz, bKash, or Nagad for authentic monetary processing.
- Cross-Platform Mobile Applications: Develop native, lightweight user extensions for iOS and Android devices utilizing cross-platform mobile frameworks like Flutter or React Native.
- GPS Route Mapping: Integrate the Google Maps API into passenger layouts to provide live transit positioning coordinates and precise arrival estimations.
- Predictive Pricing Engines: Deploy light machine learning algorithms within database structures to automate variable surge-pricing patterns during high-volume holiday periods.
- Advanced Analytics Dashboard: Build analytical charting modules featuring popularity heatmaps, financial intake timelines, and vehicle occupancy metrics for administrators.
- Progressive Web App (PWA) Evolution: Introduce dedicated client-side service workers to enable passenger access to saved tickets and offline schedule lookup behaviors.
- Scannable QR Verification: Swap standard plaintext document printing styles with unique, secure scannable QR codes generated per individual seat booking for conductor verification

References

- [1] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*, 7th Edition. McGraw-Hill Education, 2019.
- [2] Microsoft Corporation. *Microsoft SQL Server Technical Documentation*. <https://docs.microsoft.com/en-us/sql/sql-server/> Accessed: 01 June 2026.
- [3] Node.js Foundation. *Node.js Runtime Environment API Documentation*. <https://nodejs.org/en/docs/> Accessed: 01 June 2026.
- [4] Express.js Team. *Express — Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/> Accessed: 01 June 2026.
- [5] Auth0. *JSON Web Tokens (JWT) Core Specifications & Introduction*. <https://jwt.io/introduction> Accessed: 01 June 2026.
- [6] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*, 7th Edition. Pearson, 2016.
- [7] Al-Maynul Hasan. *Smart City Transport Management System (SCTMS) Source Repository*. <https://github.com/AlMaynulHasan/SCTMS> Accessed: 01 June 2026.